CSE3044 - Software Engineering Course

Problem Session

Spring 2025-2026

Notes About the Course

- TA: Res. Asst. Mehmet Kaya
  - Email: mehmet.kaya@marmara.edu.tr
  - Room: M2-201
  - Office Hours
    - Monday 13:00-15:00
    - Tuesday 9:30-11:30
- Canvas Instructure
  - Announcements
  - Lecture slides
  - Assignments
  - Submission

PS Content

1. Introduction to UML
2. Use Case Diagrams
3. Class Diagrams
4. Sequence and Communication Diagrams
5. Activity and State Diagrams
6. Discussions about lecture topics

Semester Project Guide

1. Purpose of the Project

In this course, you will design and build a software system **of your own choosing**.
The goal is not to build the “largest” or “most impressive” system, but to demonstrate that you can:

- Analyze requirements
- Make and justify engineering decisions
- Design a coherent system
- Implement core functionality
- Test what you build

This project evaluates **software engineering skills**, not just coding ability.

2. Project Topic Selection (Very Important)

You are free to choose your project topic, **within the constraints defined below**.

2.1 What Kind of Projects Are Allowed?

Your project must satisfy **all** of the following:

- ✅ Be a **software system**, not just an algorithm
- ✅ Have **at least two distinct user roles**
- ✅ Include **persistent data** (e.g., database, file storage)
- ✅ Contain **non-trivial business logic**
- ✅ Be testable (logic that can be unit- and integration-tested)

2.1 What Kind of Projects Are Allowed?

Examples of acceptable topics:

- Reservation or booking systems
- Task or workflow management tools
- Educational platforms
- Simple e-commerce systems
- Campus or organization management tools
- Smart SW systems (i.e., above systems with learning or AI capabilities)

2.2 What Kind of Projects Are NOT Allowed?

- ❌ Single-page or static applications
- ❌ Pure CRUD demos with no real logic
- ❌ Games focused mainly on graphics or physics
- ❌ Pure data science / ML projects
- ❌ Projects that are only APIs with no defined users
- ❌ Projects whose complexity is mostly in UI design

If your system can be fully described as “just forms and a database,”
it is **not acceptable**.

2.3 Scope Limits (Critical)

Your system must be:

- **Large enough** to justify design decisions
- **Small enough** to be implemented and tested within one semester

To ensure this, your project may typically include:

- ✓ 5–8 functional requirements
- ✓ 3–5 non-functional requirements
- ✓ 4–6 major system components/modules

Excessively large or vague projects will be rejected.

3. Use of AI Tools

You are allowed to use AI tools during **all phases** of the project.
However:

> AI can help you generate ideas,
>
> but it cannot justify your decisions.

You must:

- Declare where AI was used
- Critically evaluate AI suggestions
- Be ready to prepare a correcting/better alternative for each suggestion of AI
- Take full responsibility for the final system

Blindly following AI output will result in **low marks**, even if the system works.

4. Required Project Artifacts

Your project will be developed in **four main phases**, each producing a graded artifact.

- Requirements Specification Document (RSD)
- Design Specification Document (DSD)
- Implementation
- Testing Document

4.1 Requirements Specification Document (RSD)

This document defines **what the system should do**.
It must include:

- Project overview and motivation
- Stakeholders and user roles
- Functional requirements (clear, testable)
- Non-functional requirements
- Assumptions and constraints

4.1 Requirements Specification Document (RSD) - 2

Requirements must be:

- Precise
- Testable
- Consistent

Vague or aspirational requirements will be penalized.

4.2 Design Specification Document (DSD)

This document explains **how the system is designed**.
It must include:

- System architecture
- Major components/modules
- Interfaces between components
- Data model
- Key design decisions and alternatives

Diagrams must be supported by **written explanations**.

4.3 Implementation

You must implement the **core functionality** of your system.
Expectations:

- Clean and readable code
- Logical module boundaries
- Basic error handling
- Meaningful commits

A partially implemented but well-designed system can score higher than a poorly designed full system.

4.4 Testing

You must demonstrate that:

- Your system can be tested
- You understand what and why you are testing

Minimum expectations:

- Unit tests for core business logic
- At least one integration test
- A short test plan describing:
  - What is tested
  - What is not tested (and why)

Tests that merely check “does it run” are not sufficient.

5. Decision Log (Required Across All Phases)

Throughout the project, you must maintain a **Decision Log**.
Each entry must include:

1. Decision made
2. Alternatives considered
3. Rationale
4. Trade-offs
5. Risks or technical debt

5. Decision Log (Required Across All Phases) - 2

This log applies to:

- Requirements decisions
- Design choices
- Technology selection
- Testing strategy

6. Defense / Reflection

You will be required to defend your project.
You should be prepared to answer:

- Why this project scope is appropriate
- Which requirements were hardest to define
- Which design decision you would change
- What your tests do *not* cover
- Which AI suggestion you rejected

Inability to defend decisions will significantly affect your grade.

7. Project Grading

Component	**Weight**
RSD	15%
DSD	15%
Implementation	15%
Testing	15%
Project Quizzes & Defense	40%
Total	**100%**

* Tentative

8. Due Dates

Fri Feb 27, 2026	Form Your Team
Fri Mar 6, 2026	Term Project Proposal
Fri Mar 13, 2026	Editing / modifications based on instructor's & TA's comments and corrections on project proposals (if necessary including project title)
Fri Mar 27, 2026	Requirement Specification Document
Fri April 24, 2026	Design Specification Document
Thu April 30, 2026	Project Quiz 1
Fri May 29, 2026	Testing Document & Project Submission
Thu June 4, 2026	Project Quiz 2
June 1-12, 2026	Project Presentations

9. Proposal

1. Form your team (3 students for each group)
    - If you do not have a group or have missing team member(s), e-mail me; I will form the groups.
    - Equal individual effort expected.
2. Find a project topic
    - Novel content as much as possible.
    - Complexity of project counts.
3. Team meeting
    - discuss and understand the project,
    - do a brainstorming about what properties the project & system could have.
4. Name your project. Please use your creativity.
    - At most one sentence with few words, preferably a word or two words
5. Write 1-2 paragraphs that summarize your project's aim & properties.

10. Final Remarks

This project is designed to reflect **real software engineering practice**:

- Requirements are imperfect
- Design involves trade-offs
- Code is never final
- Tests are never complete

What matters is whether you can **reason about your system** and **defend your decisions**.